

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA17

ANALYTICAL PROBLEM SOLVING

1. Water Jug Problem

There are two unmarked jugs available — one holding 4 gallons and the other holding 3 gallons — along with a water pump that can refill either jug. The only actions allowed are filling a jug completely, emptying a jug on the ground, and pouring water from one jug into the other. Using only these actions, the goal is to end up with exactly 2 gallons of water sitting in the 4-gallon jug.

Assumptions considered: a jug may be topped up at the pump, any jug may be poured out onto the ground, water may be transferred between the two jugs, and no other measuring tools exist besides the two jugs.

Solution Steps

1. Fill the 3-gallon jug completely. State: (4G = 0, 3G = 3)
2. Transfer all of it into the 4-gallon jug. State: (4G = 3, 3G = 0)
3. Fill the 3-gallon jug once more. State: (4G = 3, 3G = 3)
4. Pour from the 3-gallon jug into the 4-gallon jug just until the 4-gallon jug is full. Only 1 gallon is needed, leaving 2 gallons behind. State: (4G = 4, 3G = 2)
5. Empty the 4-gallon jug completely onto the ground. State: (4G = 0, 3G = 2)
6. Pour the 2 gallons remaining in the 3-gallon jug into the 4-gallon jug. State: (4G = 2, 3G = 0)

Summary Table

Step	Action Taken	4-Gallon Jug	3-Gallon Jug
1	Fill the 3-gallon jug completely	0	3
2	Empty the 3-gallon jug into the 4-gallon jug	3	0
3	Fill the 3-gallon jug again	3	3
4	Top up the 4-gallon jug from the 3-gallon jug	4	2
5	Empty the 4-gallon jug onto the ground	0	2
6	Move the leftover 2 gallons into the 4-gallon jug	2	0

Outcome: the 4-gallon jug ends with exactly 2 gallons, while the 3-gallon jug is empty — meeting the target.

2. Mars Rover as an Intelligent Agent

A Mars Rover is an unmanned vehicle deployed on Mars to study the planet's terrain, gather rock and soil samples, photograph the surroundings, and relay findings to Earth. Because the distance between Earth and Mars introduces significant communication lag, the rover cannot be driven in real time from mission control — it must instead sense conditions around it, reason about what to do, and carry out actions largely on its own.

i. Percepts of the Agent

Percepts are whatever the rover senses from its surroundings at any given moment. For a Mars Rover these include:

- Camera imagery of the terrain
- Chemical composition readings of rocks and soil
- Ambient temperature
- Atmospheric pressure
- Wind speed and weather patterns
- Obstacle-distance readings from onboard sensors

- Terrain shape and surface texture
- Remaining battery charge and power state

In the same way a person relies on sight and hearing to understand their surroundings, the rover relies on its cameras and instruments to build a picture of the Martian environment, which then informs its next move.

ii. Nature of the Operating Environment

The environment the rover operates in can be described along several dimensions:

1. Partially observable — sensors reveal only the immediate surroundings, not the entire planet at once.
2. Dynamic — dust storms, shifting terrain, and changing weather can alter conditions at any time.
3. Sequential — earlier actions shape what is possible later; where the rover drives now affects what it can reach next.
4. Continuous — position, distance, and time all vary smoothly rather than in fixed steps.
5. Single-agent — the rover works alone, without other competing agents in its environment.

Mars presents an unfamiliar and demanding setting, so the rover has to keep adjusting its behaviour as conditions shift during its mission.

iii. Actions Available to the Agent

Typical actions the rover can carry out include:

- Drive forward
- Drive backward
- Turn left or right
- Halt movement
- Take photographs
- Gather rock samples
- Gather soil samples
- Run analysis on collected samples
- Drill into rock surfaces
- Steer around obstacles
- Transmit collected data to Earth
- Recharge via solar panels when possible

Using the information its sensors provide, the rover picks whichever action best advances its scientific objectives while keeping itself safe.

iv. Evaluating Agent Performance

A rover's performance can be judged against these measures:

1. Scientific value — how many useful samples were gathered and how good the resulting data is.
2. Ground covered — total distance travelled and area surveyed.
3. Communication success — how much accurate data reaches Earth.
4. Power management — how efficiently battery and solar energy are used.
5. Safety record — how well it avoids collisions or damage.

A rover earns a 'successful' rating when it returns valuable science, covers substantial ground, conserves energy, and stays operational and undamaged over a long mission.

v. Suitable Agent Architecture

Utility-Based Agent

A utility-based design fits the Mars Rover best.

Why this architecture: the rover is frequently faced with several possible actions and needs to pick whichever one delivers the greatest overall benefit, weighing factors such as:

- Scientific payoff
- Personal safety of the equipment
- Power cost
- Mission priorities
- Environmental hazards

For instance, when two routes are open to it, the rover favors the one that is both safer and more likely to lead to a valuable discovery.

Benefits of this approach:

- Supports more sophisticated decision-making
- Copes well with uncertain conditions
- Keeps mission goals optimized
- Balances exploration against safety

Conclusion: The Mars Rover functions as an autonomous intelligent agent — sensing its surroundings, reasoning over that information, and acting through movement, sampling, and communication. Its world is partially observable, dynamic, sequential, continuous, and single-agent. Success is measured through the science it returns, the ground it covers, its safety record, and how well it manages power, which is why a utility-based architecture suits it best.

3. The 8-Queens Problem

This is a well-known search problem in AI: eight queens must be arranged on an 8×8 chessboard so that none of them can capture another. Two queens threaten each other if they share a row, a column, or a diagonal. The configuration given in the question is not a valid solution because its first-column and last-column queens sit on the same diagonal.

Formulating the Problem

1. Initial State

An empty 8×8 board with no queens placed yet.

2. State Space

Each state describes one particular arrangement of queens on the board — which squares are occupied and which remain open for future placement. The full state space covers every possible way queens could be arranged across the board.

3. Actions (Operators)

- Place a queen on a safe square in the next available column.
- Reposition a queen if its current square creates a conflict.

As an example: place the first queen in column 1, then place the second queen in column 2 at a square that doesn't threaten the first, and continue this way until all eight queens are on the board.

4. Goal Test

A state is a goal state once all eight queens are on the board and none threatens another — meaning no two share a row, no two share a column, and no two share a diagonal.

5. Path Cost

Each queen placed is treated as a cost of 1. Since every solution requires exactly 8 placements, path cost isn't a distinguishing factor here — the aim is simply to find any arrangement that works.

Search Strategy: Backtracking

Backtracking search is the standard approach for this problem.

How it proceeds:

1. Place a queen in the first column.

2. Move on to the next column.
3. Place a queen there in a row that isn't under attack.
4. Keep going until all eight queens are placed.
5. If a column has no safe row available, backtrack to the previous queen and try a different position for it.
6. Continue this process until a valid arrangement is found.

One Valid Arrangement

Column	Row
1	1
2	5
3	8
4	6
5	3
6	7
7	2
8	4

Board layout for this solution (Q marks an occupied square):

```

Q . . . . .
. . . . . Q .
. . . . Q . .
. . . . . . Q
. Q . . . . .
. . . Q . . .
. . . . . Q .
. . Q . . . .

```

No two queens threaten one another in this layout.

Judging the Search

- Whether a valid arrangement was actually found.
- How many states had to be explored.
- Time needed to reach a solution.
- Memory consumed during the search.

Conclusion: The 8-Queens puzzle is a classic state-space search task. It starts from an empty board, progresses through queen placements, and finishes once all eight are positioned without conflict. Backtracking search handles this well, since it systematically works through placements and retreats whenever a conflict arises, avoiding wasted exploration.

4. OLA Cab Booking — Problem-Solving Agent

A customer wants to get from one place to another using the OLA app, and can pick from cab options like Micro, Mini, Sedan, Prime, or Shared depending on budget, comfort, and what's available. The app needs to guide the customer toward the option that best suits their needs and get them to their destination.

Agent Type

Goal-Based Agent

A goal-based agent design fits this scenario well, since:

- The customer's objective is simply reaching the destination.
- The app weighs up the available cab choices.
- It settles on the option that matches what the customer needs.

- It works out the sequence of actions that gets the customer there efficiently.

The booking system is therefore best described as a goal-based problem-solving agent.

Problem Formulation

Initial State

- Customer launches the OLA app.
- App detects the customer's present location.

Actions

- Type in the destination.
- Browse the cab options on offer.
- Weigh up price against comfort.
- Pick a cab category.
- Finalize the booking.

Goal State

- The customer arrives safely at the destination.

Path Cost

- Fare charged.
- Time spent travelling.
- Time spent waiting for pickup.

Pseudocode

Algorithm OLA_CAB_BOOKING

Start

1. Open OLA Application
2. Detect Current Location
3. Input Destination
4. Display Available Cab Types
 - Micro
 - Mini
 - Sedan
 - Prime
 - Shared
5. Customer Selects Preferred Cab
6. Check Cab Availability
7. IF Cab Available THEN
 - Calculate Fare
 - Confirm Booking
 - Assign Driver
 - Start Trip
 - Reach Destination
 - Display "Trip Completed Successfully"
- ELSE
 - Display "Selected Cab Not Available"
 - Prompt Customer to Choose Another Cab
- END IF

Stop

How It Works

1. Customer opens the OLA app.
2. The app pinpoints the current location.
3. The destination is entered.

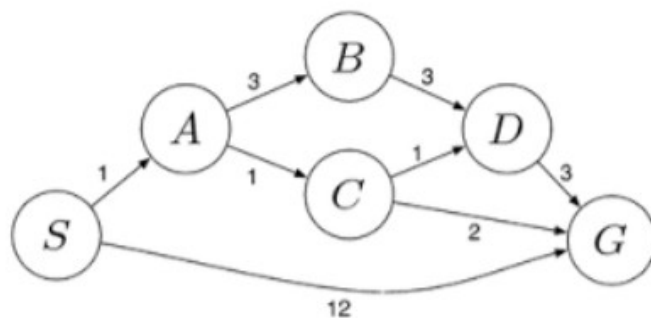
4. Available cab categories are shown.
5. Customer picks a cab based on comfort and cost.
6. The system verifies availability.
7. If available, the ride is booked and a driver assigned.
8. The customer travels to the destination.
9. The goal — arriving safely — is fulfilled.

Conclusion: The OLA Cab Booking system is best modeled as a goal-based agent because its central purpose is helping the customer reach their destination by matching them with a suitable cab and seeing the trip through to completion.

5. Least-Cost Delivery Routing with Uniform Cost Search

A logistics company needs to move packages between warehouses while keeping cost as low as possible. Every route between warehouses carries its own cost (combining fuel and time). The task is to determine the cheapest path from a starting warehouse S to a destination warehouse G, using the Uniform Cost Search (UCS) algorithm.

The delivery network below is represented as a weighted graph:



Step-by-Step Expansion

Step	Node Expanded	Cost	New Nodes Added
1	S	0	A(1), G(12)
2	A	1	B(4), C(2)
3	C	2	D(3), G(4)
4	D	3	G(6)
5	G	4	Goal Reached

Comparing Possible Paths

Path	Total Cost
S → G	12
S → A → B → D → G	10
S → A → C → D → G	6
S → A → C → G	4

Result

Item	Value
Start Node	S
Goal Node	G
Least Cost Path	S → A → C → G
Total Cost	4

Final Answer: the cheapest route is $S \rightarrow A \rightarrow C \rightarrow G$, with a total cost of $1 + 1 + 2 = 4$.